# Improving Multi-Constraint Instruction Following

*in 1B-Parameter Language Models via RECAST Fine-Tuning*


Shreyas Natarajan • M.S. Data Science
Rinkle Rose Renny • M.S. Data Science
Mark Aashish Lnu • M.S. Data Science
Gayathri Ethiraj • M.S. Data Science


***STAT 453 • Team 15 • Spring 2026***


University of Wisconsin-Madison

# Background: Why Small Models Struggle

- Frontier LLMs (70B-400B+) are powerful but expensive - high latency, high cost, hard to deploy at scale.
- Models in the 1-2B parameter range run cheaply on a single GPU and respond fast.
- But they consistently fail at constraint-following - simultaneously satisfying format, length, tone, keywords, and structure in one prompt.
- This matters in real deployments: structured reports, marketing copy, formatted API responses.

# Quantifying the Gap

- Benchmarks like IFEval and FollowBench expose this failure clearly.
- Meta has not published their 1B model accuracies which is its weakest model
- Performance degrades sharply as the number of simultaneous constraints grows.
- We know that the problem exists, not the solution

**~77.4%**

IFEval prompt-strict accuracy

*Llama-3.2-3B*

# Research Questions

1. Can RECAST-based fine-tuning meaningfully improve constraint-following in 1B-parameter models?

2. Does full fine-tuning outperform LoRA for this task?

3. Which constraint categories benefit most from fine-tuning, and which remain difficult?

# Literature Review: Benchmarks & Data

## Constraint-Following Benchmarks

- IFEval (Zhou et al., 2023): 25 verifiable instruction categories
- FollowBench (Jiang et al., 2024): multi-level constraint difficulty
- Both diagnose the problem, but neither provides training data

## RECAST Framework (Guo et al., 2025)

- Constraint back-translation: Extracts constraints from gold responses, injects them back into prompts
- Every constraint has an automated validator (rule-based or LLM judge)
- Outperforms larger instruct-tuned models at 7B-8B scale
- Untested at 1B, that is our gap

# Literature Review: Fine-Tuning Methods

| Dimension | LoRA (Hu et al., 2022) | Full Fine-Tuning |
|---|---|---|
| **Trainable params** | ~0.5% (~6M) | 100% (~1.2B) |
| **VRAM peak (1B)** | ~7-9 GB | ~16-20 GB |
| **Storage per adapter** | ~30-60 MB | ~2.4 GB |
| **Catastrophic forgetting** | Lower risk | Higher risk |
| **Studied for constraint-following?** | No | No |

*Shuttleworth et al. (2024) - LoRA introduces "intruder dimensions" affecting forgetting; tested for reasoning/code, not constraint-following.*

# Method Overview: End-to-End Pipeline

### 1. Preprocess

**Clean RECAST-30K**

9-stage pipeline

Dedup + Filter + Augmentation

**Results in 90K**

### 2. Train

**Full FT** (constraint-aware loss)

and

**LoRA** r=8 on q_proj/v_proj

### 3. Evaluate

Rule-based checker

+ **LLM judge** (style, topic)

**CSR** + **Hard CSR**

### 4. Cross-validate

k-fold split

IFEval generalization

**Output: Per-constraint pass rates, Hard CSR, and Per-category gain analysis**

# Implementation: Model & Dataset

## Base Model

- Llama 3.2 1B Instruct (1.24B params)
- Causal, decoder-only Transformer
- Loaded in bfloat16 on A100 40GB
- Gradient checkpointing for full FT
- Standard Llama-3 chat template format

## RECAST-30K Dataset

- ~30K multi-constraint instruction-response pairs
- 15 constraint categories: Length, Keyword, Start/End_With, Style, Topic, Format, Tone, Numerical, Role-playing… (includes rule-based & model-based)
    - ~13.4 constraints per instruction
- Each record includes a rule_evaluate_dict with structured validator parameters
- Modality: text only; max sequence length 512 tokens
- Wide domain coverage from real responses
- Human validation with 90.1% agreement

*For the 5-fold cross validation*
*Training/Validation/Test split: 72% / 18% / 10%*

# Sample from RECAST-30K Dataset


"prompt_winner": "qwen-plus_add_constraint_to_tulu",
"winner_prompt": "Draft a thank you letter to carter bank & trust for their exceptional financial services. the letter must be written in english and entirely in lowercase, including the name of the bank. begin your response with the word \"dear\" and adopt a heartfelt and appreciative tone throughout the response. focus on expressing gratitude for banking services and customer support, and include at least two explicit customer testimonials or quotes. limit the response to 100-200 words and ensure it consists of approximately 13 sentences. make sure the letter includes one instance of the phrase \"financial services\". also, ensure that the number of words in all capital letters does not exceed one. your response must terminate with \"Name\".",
"response_winner": "response_nano",
"winner_response": "dear carter bank & trust,\n\ni am writing to express my heartfelt gratitude for the exceptional financial services you have provided. from the moment i opened my account, your team has gone above and beyond to ensure my needs are met with professionalism and kindness. your customer support has been truly outstanding, making every interaction a positive experience. \n\nthanks to your dedicated staff, i feel confident and secure in managing my finances. one customer testimonial that resonated with me is, \"your team made me feel valued and understood during every step of my banking journey.\" another says, \"the personalized attention and prompt assistance have made all the difference.\" these words reflect the genuine care and commitment you show to your clients.\n\nyour financial services have truly made a difference in my life, and i am grateful for your ongoing support. thank you for your unwavering dedication to excellence. i look forward to continuing our relationship.\n\nsincerely, \nname",
"id": "personas_IF_se4up1py8y94phdji1w9rouy",
"response_of_winner_prompt": "dear carter bank & trust,\n\ni am writing to express my heartfelt gratitude for the exceptional financial services you have provided. from the moment i opened my account, your team has gone above and beyond to ensure my needs are met with professionalism and kindness. your customer support has been truly outstanding, making every interaction a positive experience. \n\nthanks to your dedicated staff, i feel confident and secure in managing my finances. one customer testimonial that resonated with me is, \"your team made me feel valued and understood during every step of my banking journey.\" another says, \"the personalized attention and prompt assistance have made all the difference.\" these words reflect the genuine care and commitment you show to your clients.\n\nyour financial services have truly made a difference in my life, and i am grateful for your ongoing support. thank you for your unwavering dedication to excellence. i look forward to continuing our relationship.\n\nsincerely, \nname"


8 sentences ✔, ~150 words ✔, all lowercase ✔

# Data Preprocessing: 9-Stage Pipeline

**1 Language detection**

langdetect on first 2000 chars; English only

**2 HTML / Entity removal**

Strip tags and entities via regex

**3 Emoji + control chars**

Replace emoji with spaces. Strip invisible Unicode.

**4 Quality gates**

Drop prompts < 15 words or < 85% printable

**5 Constraint integrity**

Validate added_constraint dict & counts

**6 Stopword removal**

On prompt only, after length gate

**7 Exact dedup**

SHA-256 fingerprints of normalized prompts

**8 Near dedup**

MinHash LSH, Jaccard $\tau = 0.85$, 128 perms

**9 Augmentation**

- Lexical editing (Synonym replacements, Dropping non-constraint words) -> ~30K
- Back translation technique (English <-> German) -> ~30K
- Total = ~90K

# Full Fine-Tuning

## Full Fine-Tuning Approach

- 4 baseline models (proposal): Llama 3.2 (1B), Gemma 3 (1B), Qwen 2.5 (1.5B), SmolLM2 (1.7B)
- Models vary in size (1B–1.7B) → analyze impact of scale

## Training Setup

- All parameters updated (~1.24B)
- Introducing constraint-aware penalty

## Constraint-Aware Loss

- This penalty is based on how well the model's output satisfies predefined constraints
- Implement one validator per constraint type
- each function takes the model's output and returns a simple pass or fail.
- Evaluate constraint satisfaction across multiple categories

# LoRA Fine-Tuning

## LoRA (Low-Rank Adaptation)

- LoRA freezes the base model and introduces small trainable matrices into parts of the network.
- Target modules: q_proj, v_proj on every attention layer
- Rank $r = 8$; $\alpha = 2r$; dropout = 0.05
- Saves only the adapter , base remains frozen

## Training Setup

- 81500 Parameters are trained in 1.23 B parameter Model
- Introducing constraint-aware penalty

### Experiment Goal

- Compare with full fine-tuning
- Same setup across models for fairness

# Evaluation: Hybrid Rule-Based + LLM Judge

## Rule-Based Mode (default)

- Deterministic checks from rule_evaluate_dict:
  - length - word count within [min, max]
  - keyword - required keywords appear ≥ N
  - start_with - response begins with string
  - end_with - response ends with string
- Returns 0 / 1 per constraint
- ConstraintChecker class supports 18 types total

## LLM-Judge Mode

- Covers qualitative constraints - style, topic
- Default judge: Qwen2.5-7B-Instruct
- Optional: Gemma-2-9B-Instruct (4-bit quantized)
- Outputs JSON verdict per constraint
- Falls back to rule-based on parse failure

# Scoring: Constraint Satisfaction Rate

**Per-example CSR**

$$\text{CSR}^{(i)} = \text{satisfied_i} / \text{total_i}$$

*Soft score: fraction of constraints satisfied*

Range: [0, 1]

**Hard CSR**

$$\text{Hard CSR} = \mathbb{1}[\text{all constraints satisfied}]$$

*Strict score: 1 only if every constraint passes*

Range: {0, 1}

**Why both?**

- Soft CSR captures partial credit - useful during training to see where progress lands.
- Hard CSR matches user expectations - a response failing one constraint is usually unacceptable in deployment.
- Per-constraint pass rates surface category-specific weaknesses (e.g. length).

# Results: Baseline

```
Base Model CSR: 28.9%
Per-type scores:
  word_length: 32.0%
  sentence_length: 100.0%
  format: 100.0%
  keyword: 40.0%
  start_with: 56.5%
  end_with: 36.0%
  all_upper: 100.0%
  all_lower: 100.0%
  no_commas: 100.0%
```

# Key Findings & Design Decisions

### LoRA gave better overall results and lower cost

Surprisingly LoRA gave us better results, taking lesser time and lesser compute.

**LoRA >> Full fine-tuning**

### Full Finetuning took more GPU and cost

Full-finetuning did improve the model but not by much and it took a lot of compute and around 4.5 hours to finetune. We believe that it overwrote over all the initial learning. Maybe because of catastrophic failure. For LoRA, we trained only a fraction of the total parameters.

### Loss Function Testing

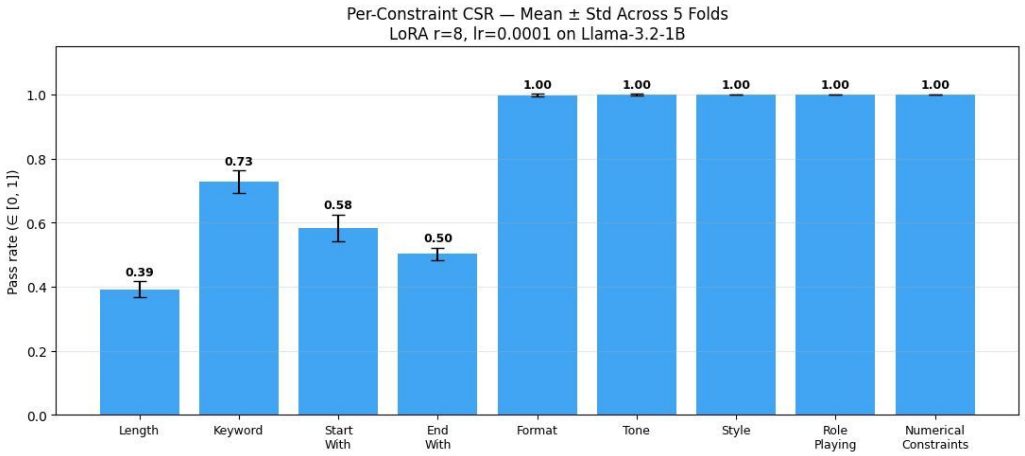
With the time that we had, we tested out different loss functions - (1-score/total) and cross entropy loss. Our first custom method was good and straight-forward but we had some difficulties with backpropagation. CE loss is a probabilistic measure and it seemed to work better for our gradient-based optimization.

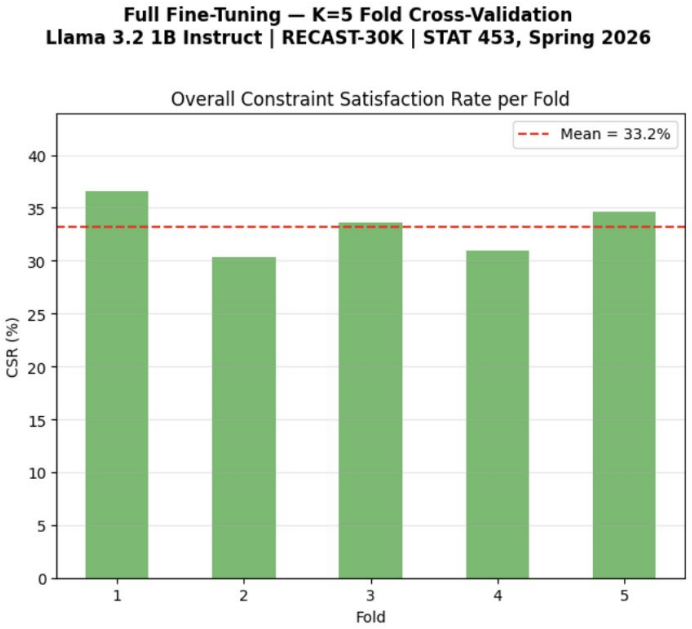
# Key Findings & Design Decisions

## LORA FINETUNING

```
==================================
 FINAL 5-FOLD LORA CSR = 0.8000
==================================

Per-fold:
 fold     csr   n
   1 0.797222 200
   2 0.797222 200
   3 0.796667 200
   4 0.803889 200
   5 0.805000 200
```




## FULL FINETUNING

```
==================================================================
K-FOLD RESULTS (K=5)
==================================================================
Fold 1: eval_loss=1.4217  train_loss=0.0000  CSR=36.6%
Fold 2: eval_loss=1.4215  train_loss=10.9864  CSR=30.4%
Fold 3: eval_loss=1.4177  train_loss=10.9924  CSR=33.6%
Fold 4: eval_loss=1.4251  train_loss=10.9826  CSR=31.0%
Fold 5: eval_loss=1.4229  train_loss=10.9752  CSR=34.6%

Eval Loss  - mean=1.4218  std=0.0024
Train Loss - mean=8.7873  std=4.3937
CSR        - mean=33.2%  std=2.3%
==================================================================
```

# Computational Cost

**Hardware** Google Colab Pro+ • A100 GPU

| Run Type | VRAM Peak | Estimated Time | Trainable Params |
|---|---|---|---|
| LoRA (rank=8, 1B) | 7-9 GB | 2-4 hr | 6.2M (0.5%) |
| Full FT (1B) | 16-20 GB | 5-8 hr | 1.24B (100%) |

# Limitations

## Compute Cost

Compute Cost and Budget was our biggest issue since we needed bigger GPU to run our finetuning faster. Using Google Colab Pro's A100 GPU, it took us around 4 hours to fully finetune the model

## 2 Models Tested

Proposal targeted four 1-2B models; resource constraints narrowed primary experiments to Llama 3.2 1B Instruct and Qwen 2.5 1.5B Instruct.

## Judge bias

Using a 7B model as judge for 1B outputs introduces systematic bias - judge standards may differ from human annotators.

# Future Work & Takeaways

## Future Work

- Extend to Gemma 3 1B, SmolLM2 1.7B - multi-model robustness check.
- Explore higher LoRA ranks (r=16, 32) for harder qualitative constraints.
- Investigate why our experimentation showed that LoRA did better.
- Per-category targeted training data augmentation (length and style - heavy mini-batches).
- Work with a bigger dataset and better GPU to better finetune the model.

## Takeaways

- End-to-end pipeline for constraint-following at 1B scale: preprocess → train → eval → CV.
- Modular checker supports 9 constraint types.
- LoRA performed surprisingly better than Full Finetuning

https://github.com/shreyasnat2804/stat-453-constraint-based-llm/tree/main

***Thank you***